Lecture 17
Functional synthesis and Verification

Armando Solar-Lezama

Moving beyond Inductive synthesis

We want to synthesize functions that satisfy richer correctness criteria

Key questions

- Specification formalisms
  - How do we describe the intended behavior?
- How do we ensure correctness
  - No longer trivial as in the inductive case
- Synthesis approaches
  - Deductive vs. inductive

Defining correctness

Attempt 1: Not enough

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i \,.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out\left[i+1\right]$

$len(out) = n$

Defining correctness

Attempt 2: Still not enough

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i \,.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out\left[i+1\right]$

$$len(out) = n$$

$$\forall i \,.\, 0 \leq i < n-1 \Rightarrow \exists j \; in\left[j\right] = out[i]$$

Defining correctness

Attempt 3: Still not enough

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i \,.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out\left[i+1\right]$

$len(out) = n$

$\forall i \,.\, 0 \leq i < n-1 \Rightarrow \exists j \; in\left[j\right] = out[i]$

$\forall i \,.\, 0 \leq i < n-1 \Rightarrow \exists j \; in[i] = out[j]$

Defining correctness

Attempt 4: Is it enough?

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i \,.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out[i+1]$

$len(out) = n$

~~$\forall i \,.\, 0 \leq i < n-1 \Rightarrow \exists j \; in[i] = out[i]$~~

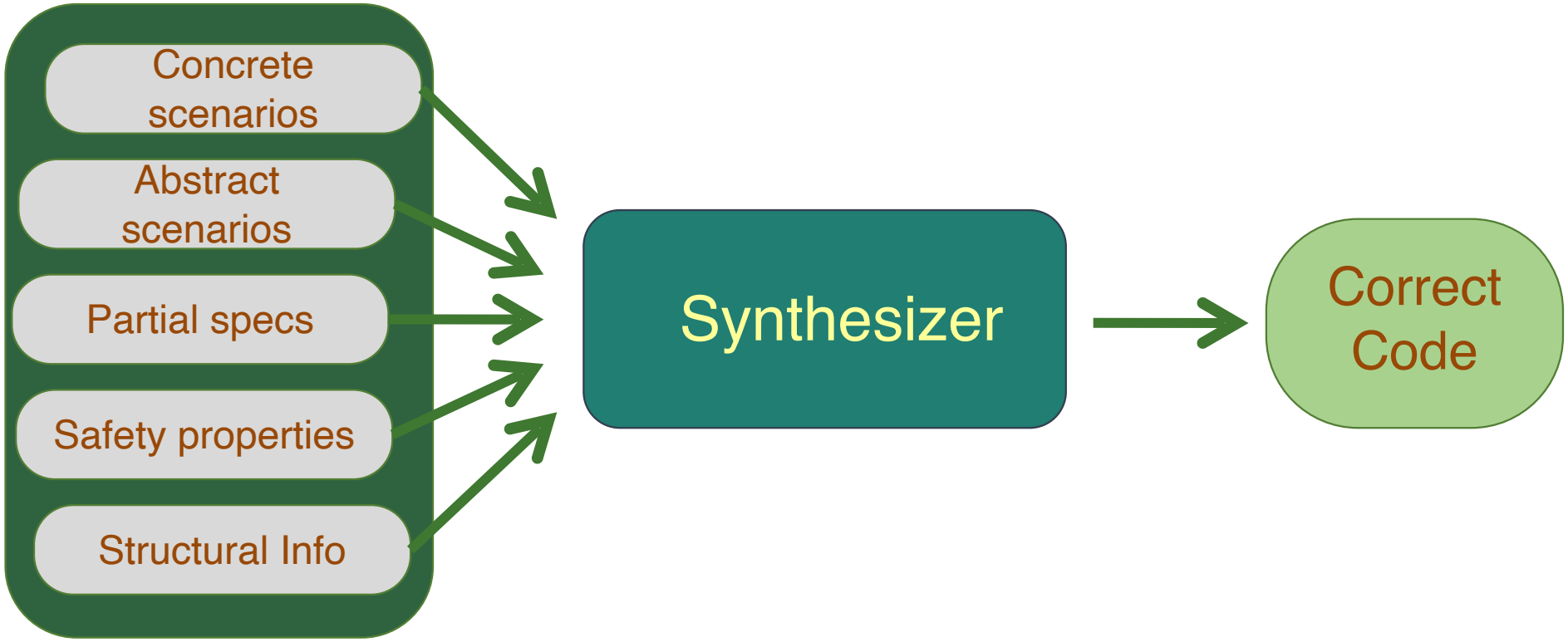
$\forall i \,.\, 0 \leq i < n-1 \Rightarrow \exists j \; in[i] = out[j]$

$\exists p : int \rightarrow int \,.\; \forall i \,.\; i < n \Rightarrow \; out[i] = in[p(i)] \wedge \exists j \in [0,n) \,.\; p(j) = i$

Multimodal Synthesis

Trick:

combine many simple specs in different formalisms to fully constrain the behavior



Ensuring correctness

This is a hard problem in general

$$\forall\ in \quad \left(in,\ P \models Spec\right)$$

Two points of view:

- Not my problem
  - This is a synthesis class, not a verification class
- Synthesis can make verification simpler
  - Synthesize code that is easier to prove correct
  - Ensure correctness by construction

Constraint-based verification

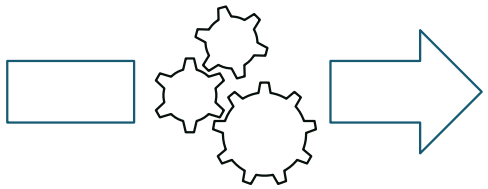
Code

```
if(x > y){
  t = x - y;
  while(t > 0){
    x = x - 1;
    y = y + 1;
    t = t - 1;
  }
  assert y=x_in
  assert x=y_in
}
```



Verification Condition Generation

$$\forall in \;.\; Q(in)$$



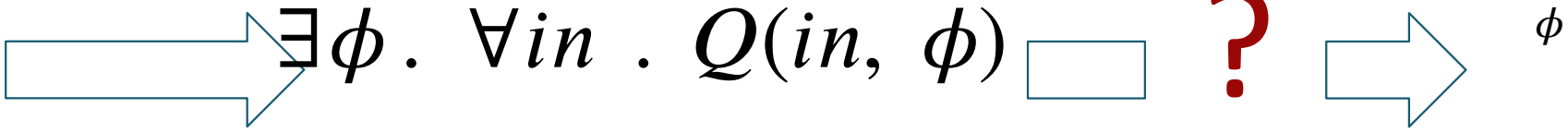
Solver

OK

… and Synthesis

Code

```
if(x > y){
  t = x - y;
  while(t > 0){
    x = x - ϕ(??1);
    y = y + ϕ(??2);
    t = t - ϕ(??3);
  }
  assert y=xin
  assert x=yin
}
```



Verification Condition Generation

Historically, solvers that can deal with quantifier alternations have not been very scalable

Key challenges

How do we generate the predicates Q

- Tradeoffs between efficiency, completeness and soundness

How do we deal with quantifier alternation

- Quantifier elimination
- Abstraction
- CEGIS

Quantifier Elimination

Use algebraic equalities manipulation to derive relationships between unknowns

- powerful, but limited in scope and potentially expensive

```
foo(x,y){
  t = y*φ(??1)
  v = x*φ(??2)
  assert t*x + v*y = 0;
}
```

$$\exists \phi \ \forall x, y \ \mathrm{x*y*}\phi_1 + \mathrm{y*x*}\phi_2 = 0$$

$$\Rightarrow \phi_1 = -\phi_2$$

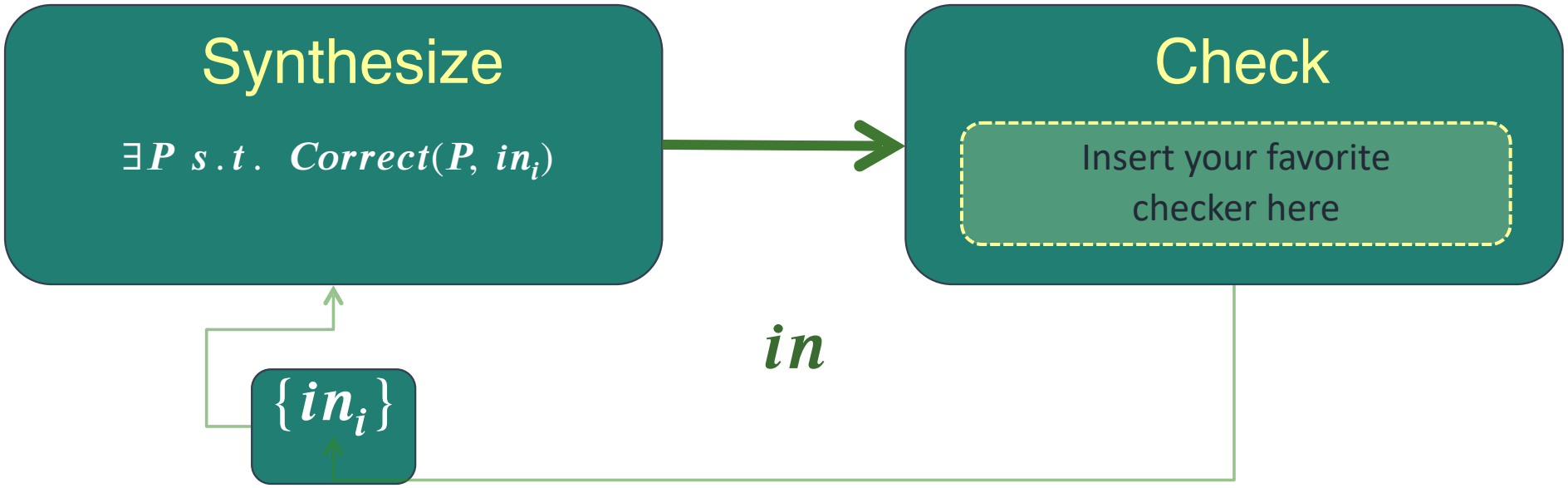
CEGIS

Counterexample guided inductive synthesis

Ideas

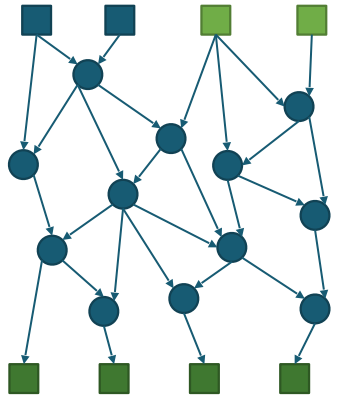
- Rely on an oracle to tell you if your program is correct
- If it is not, rely on oracle to generate counterexample inputs
- Reduce to an inductive synthesis problem

CEGIS


Synthesize
$\exists P \; s.t. \; Correct(P, in_i)$
Check
Insert your favorite checker here
in
$\{in_i\}$

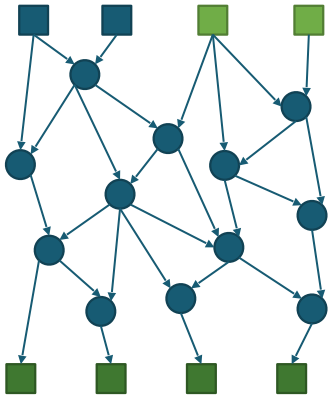
Constraint-based CEGIS

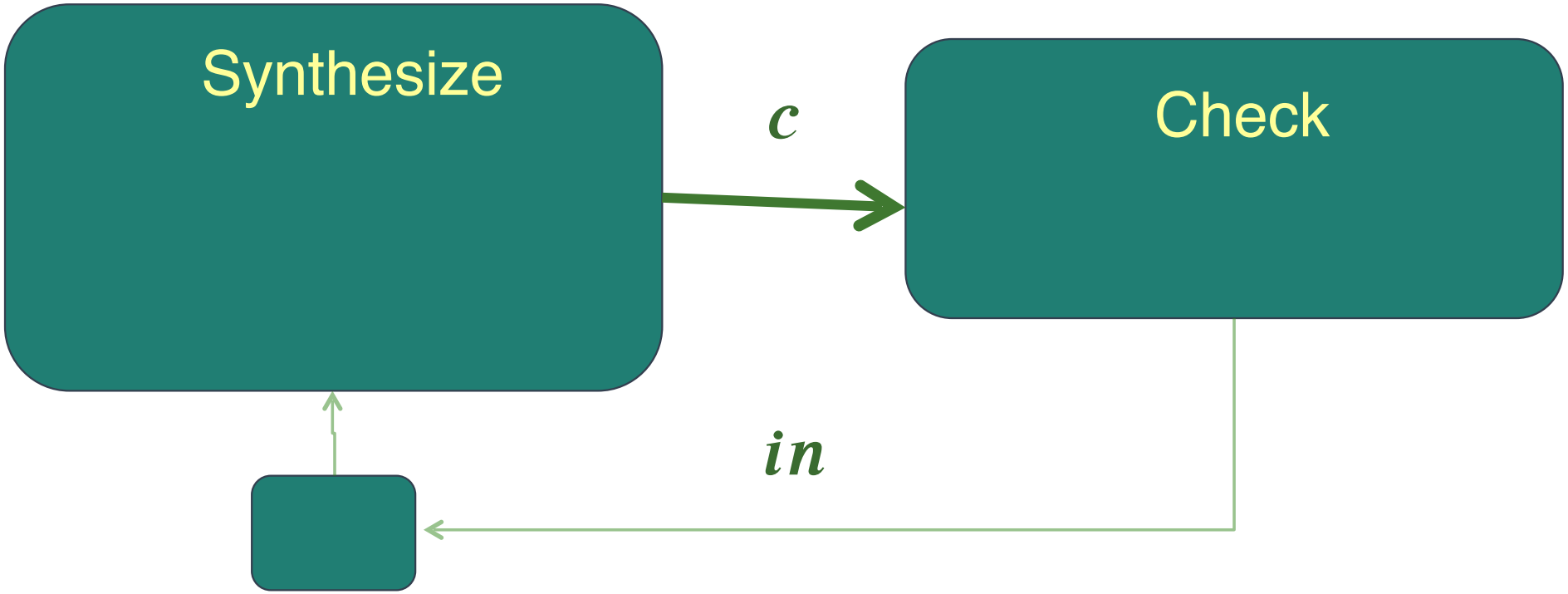
$$Q(c,\ in)$$



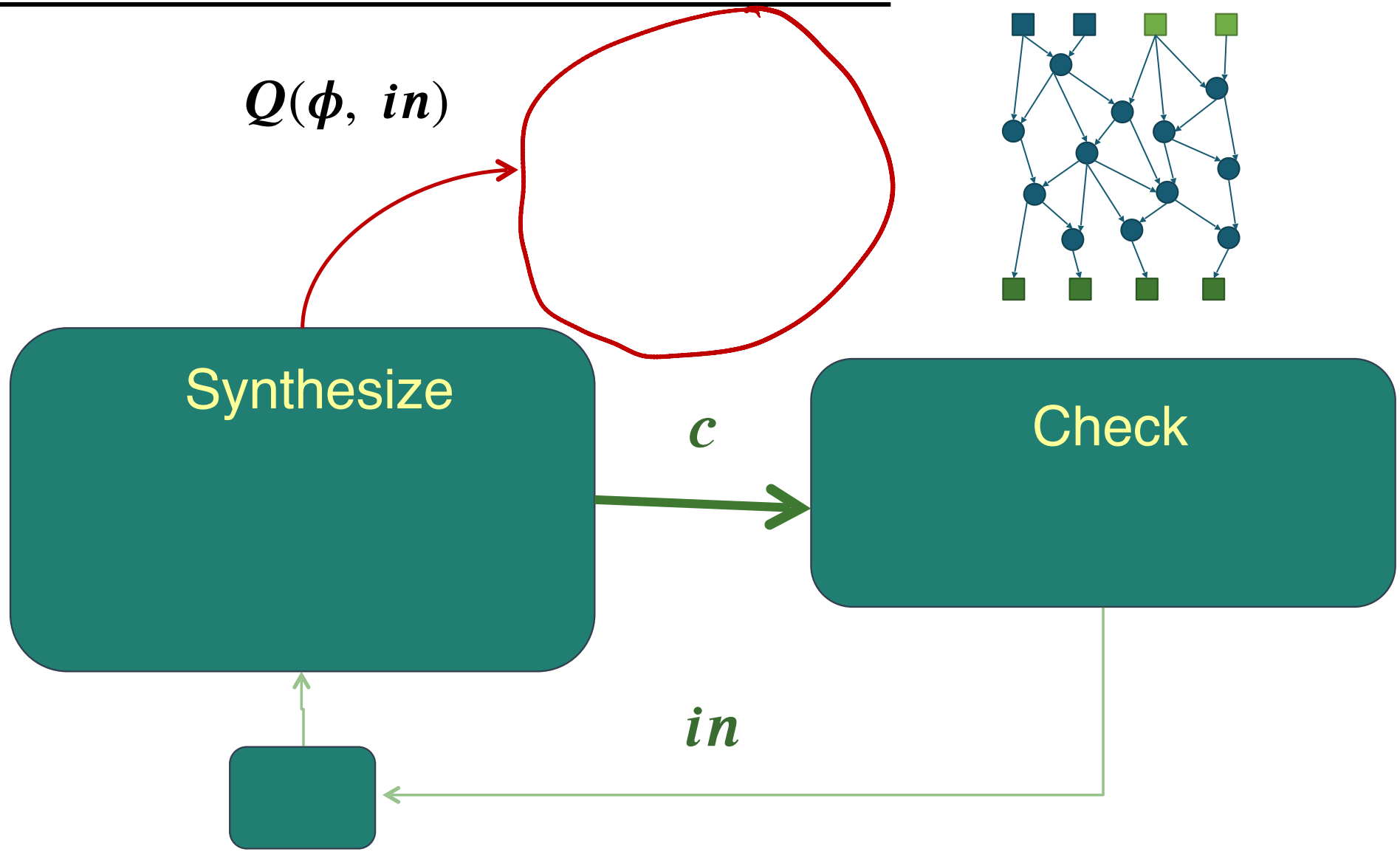
Constraint-based CEGIS

$$Q(\phi,\ in)$$

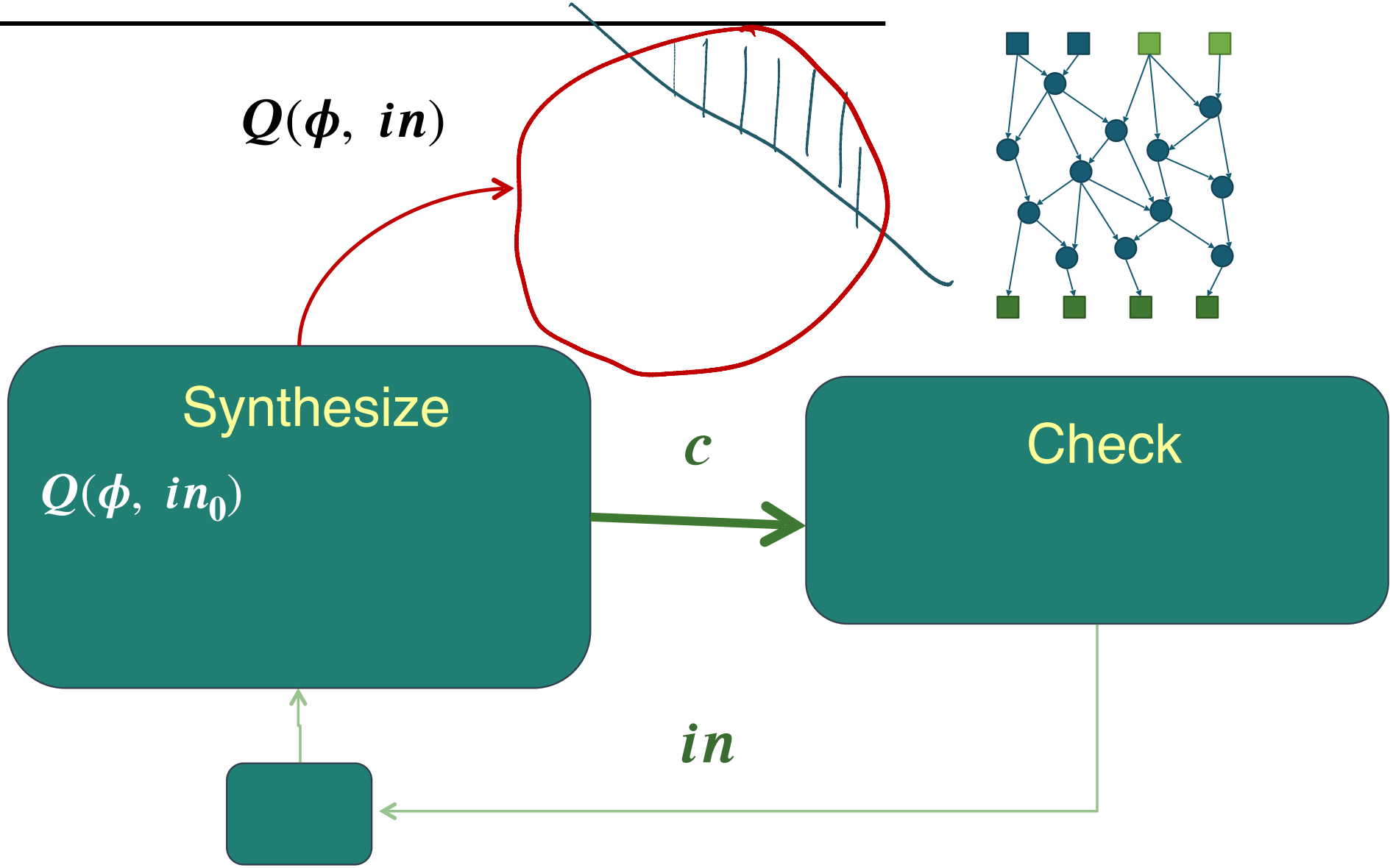




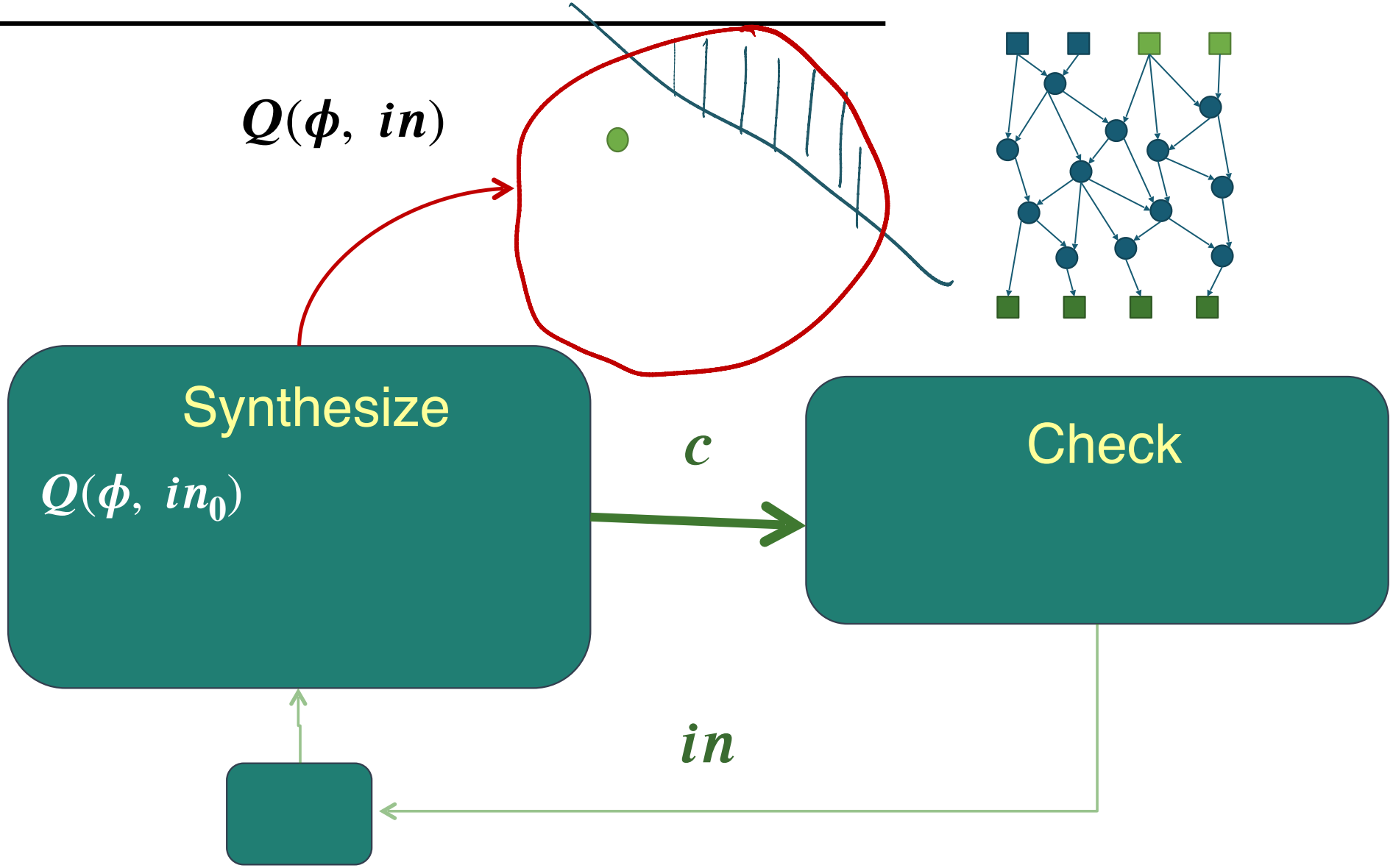
Constraint-based CEGIS



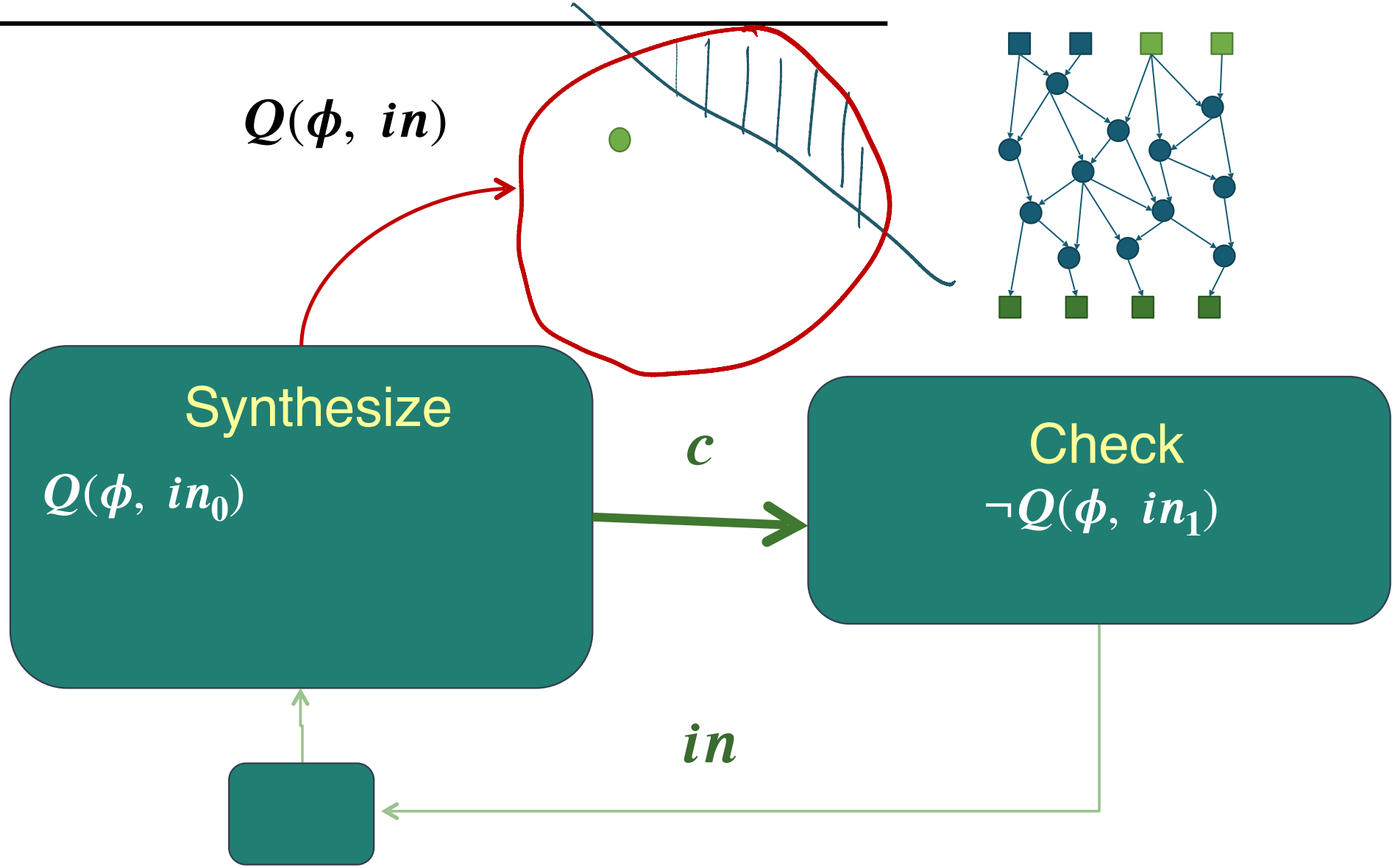
Constraint-based CEGIS



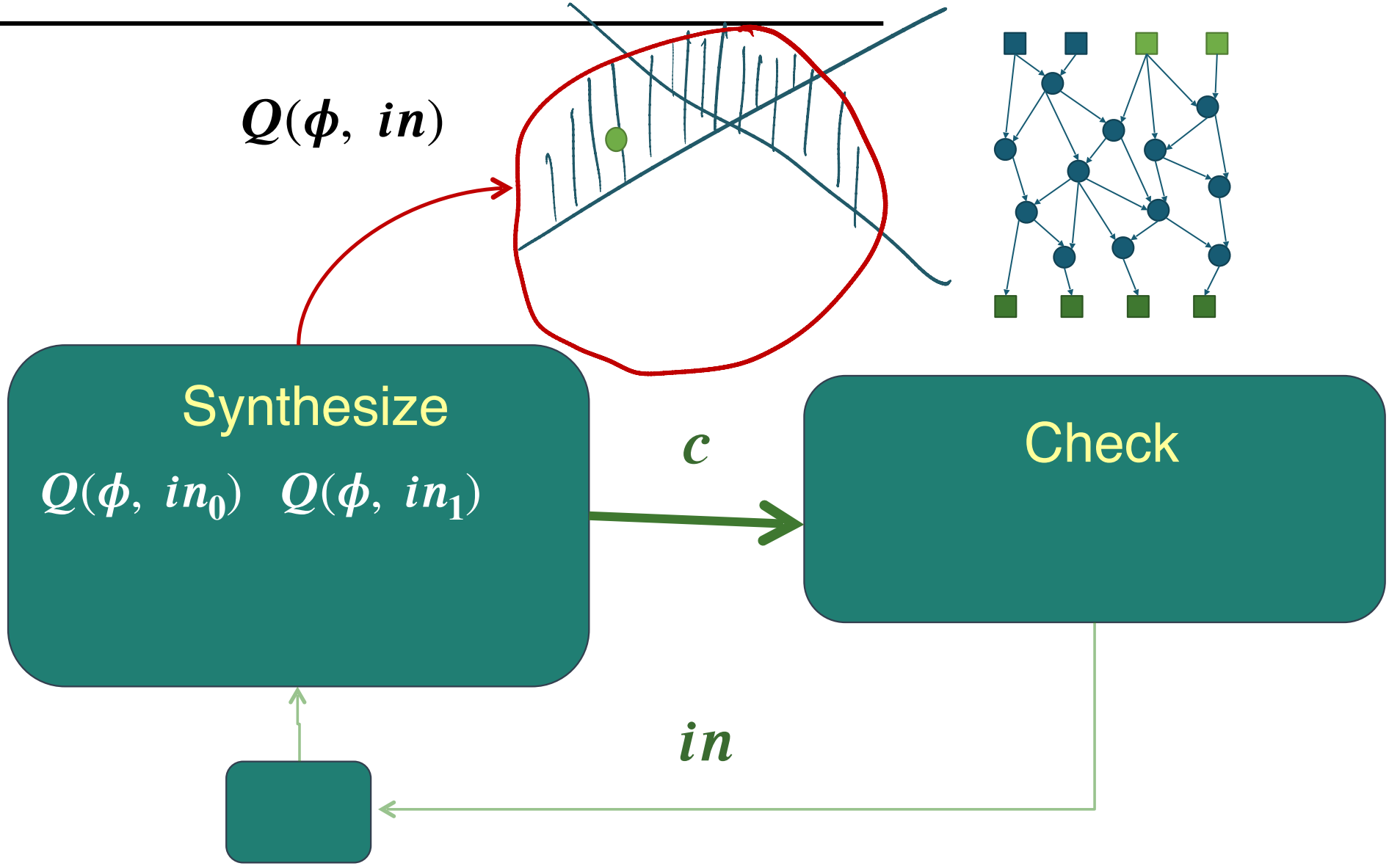
Constraint-based CEGIS



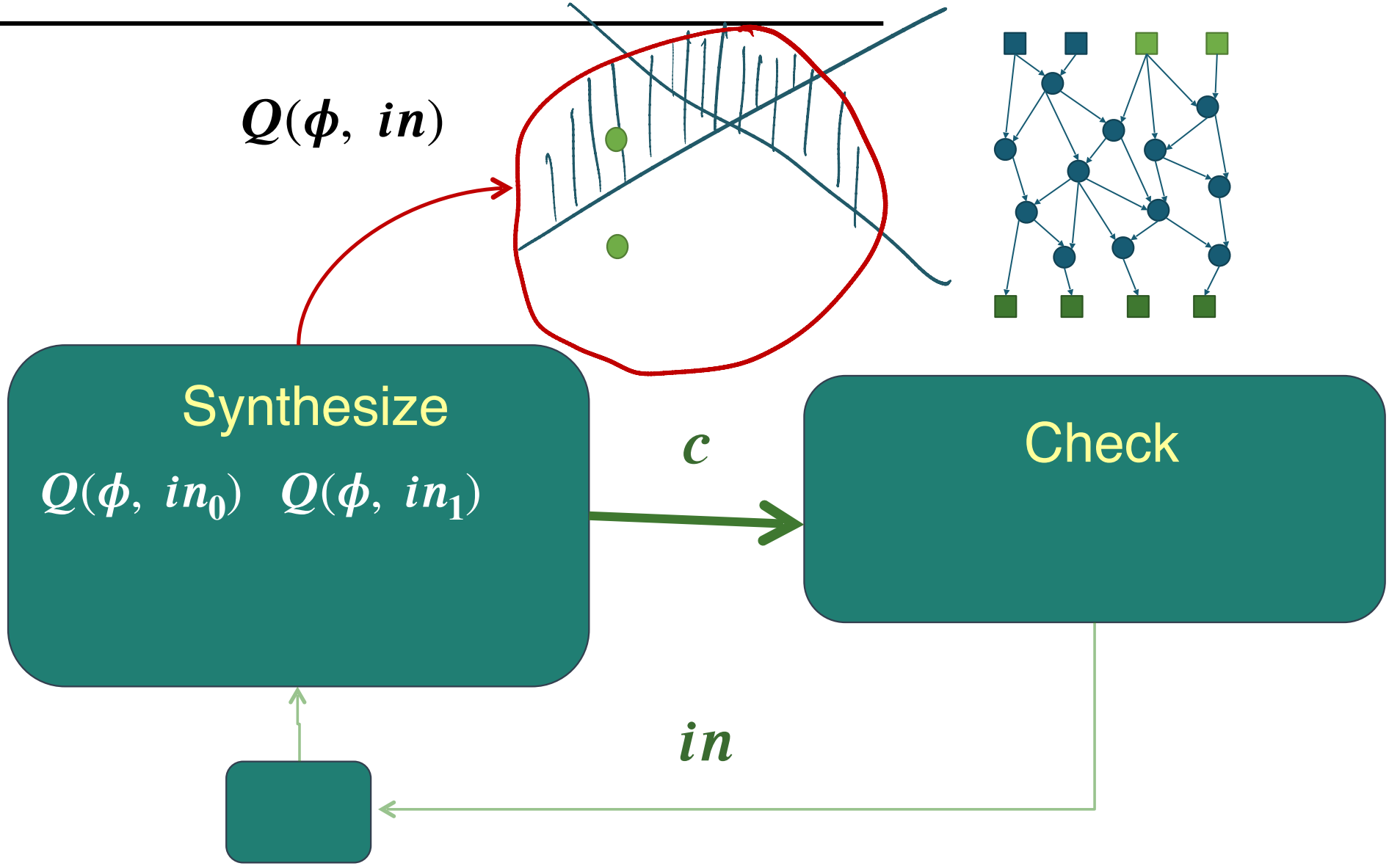
Constraint-based CEGIS



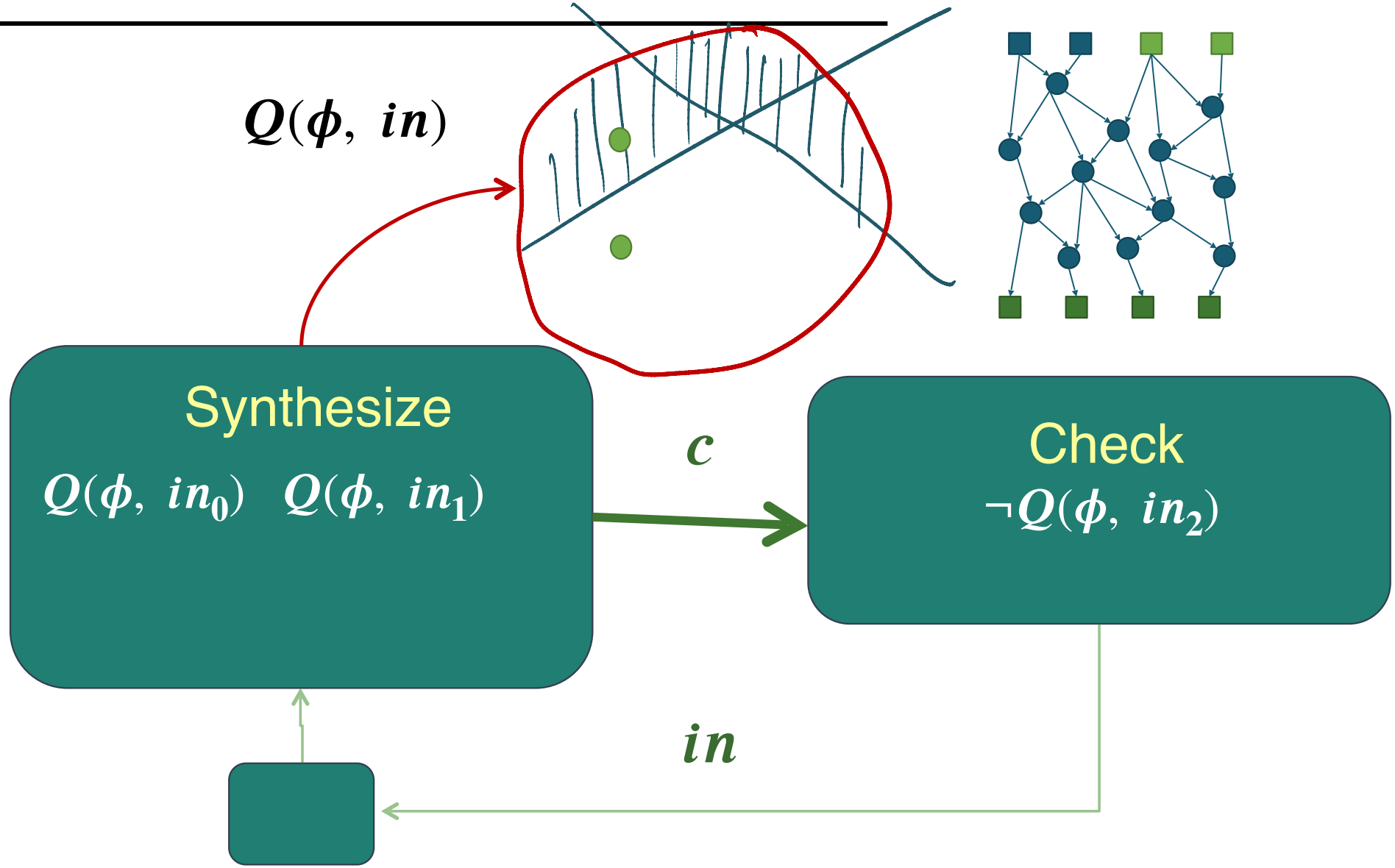
Constraint-based CEGIS



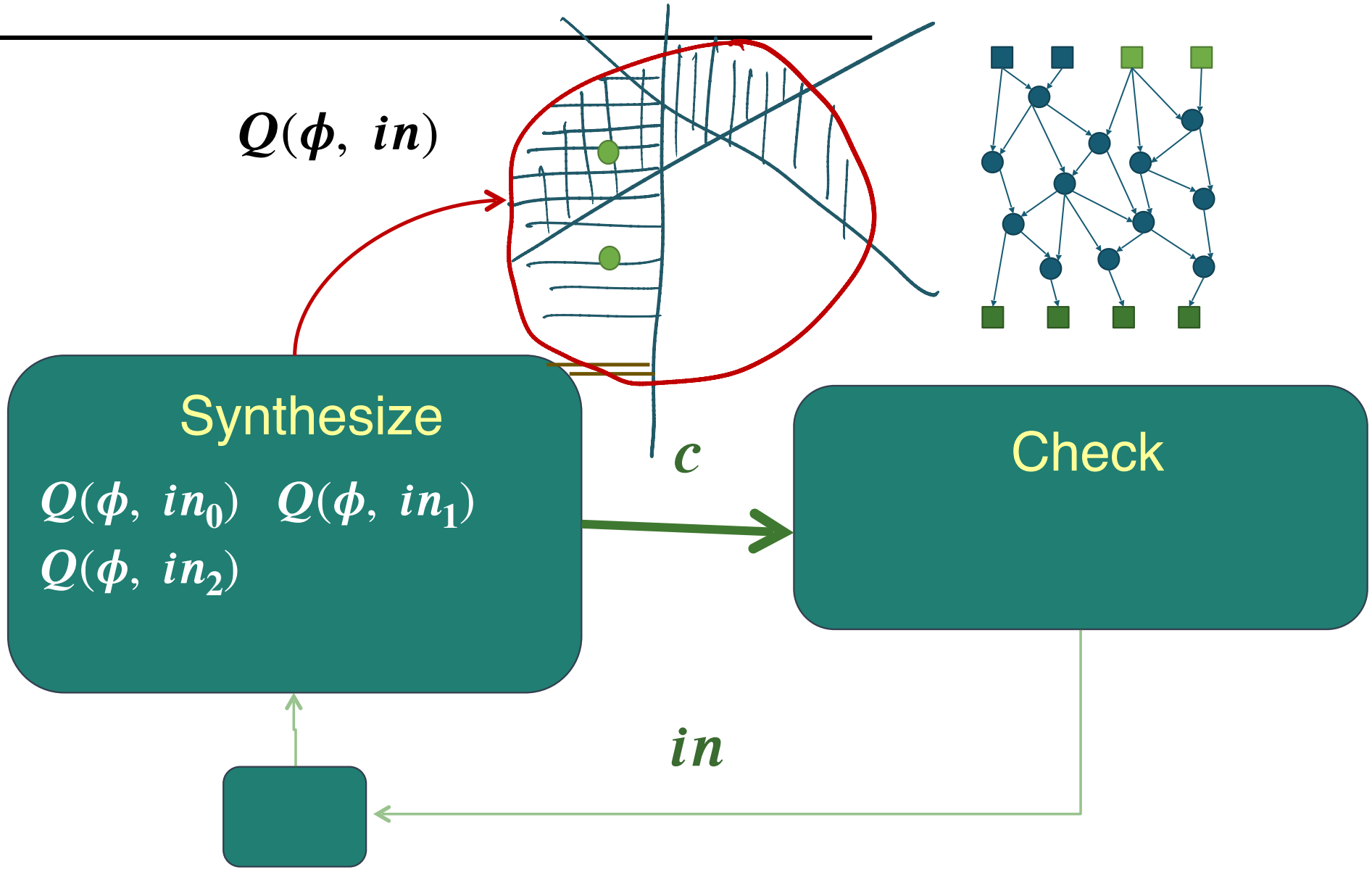
Constraint-based CEGIS



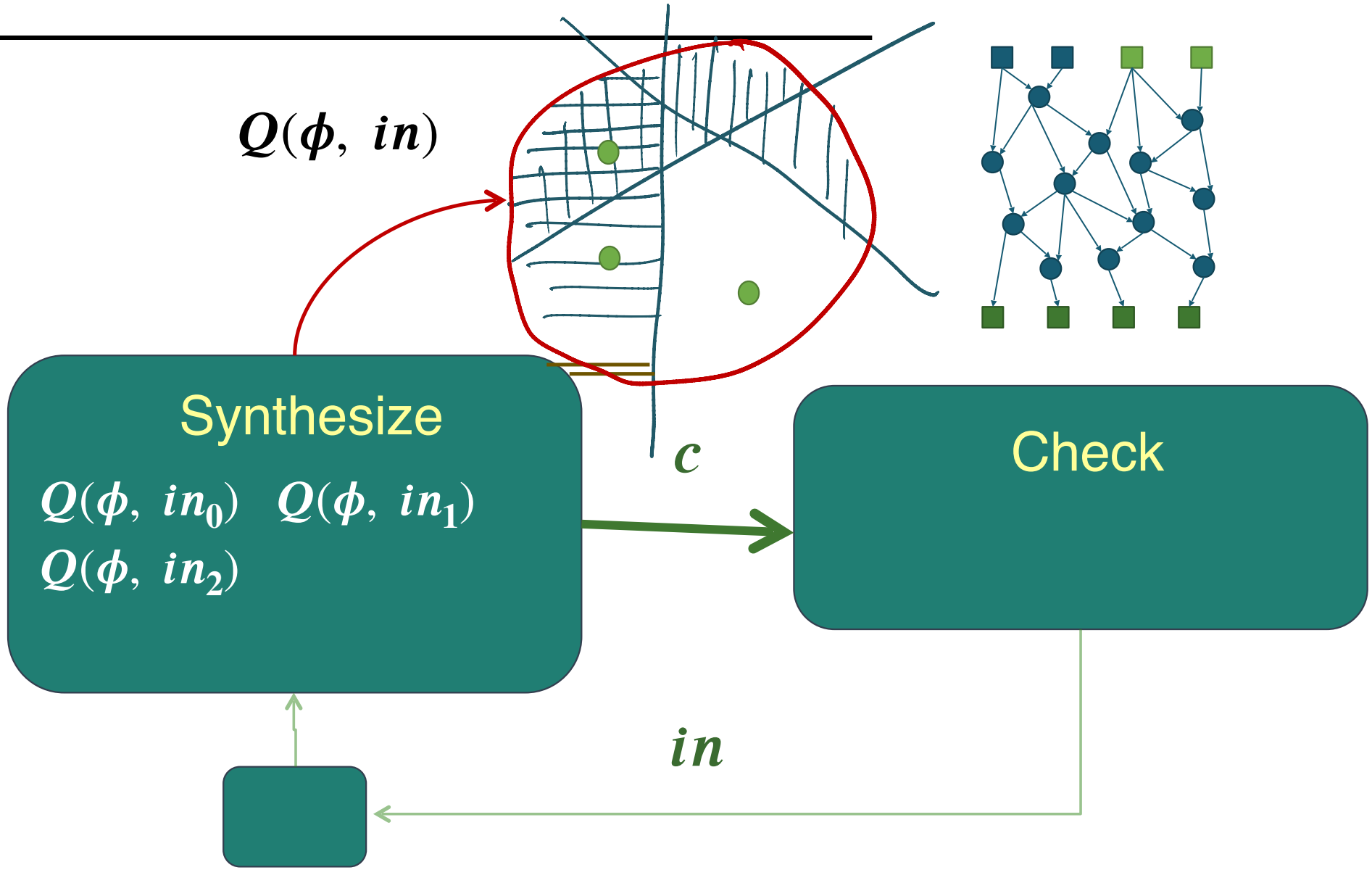
Constraint-based CEGIS



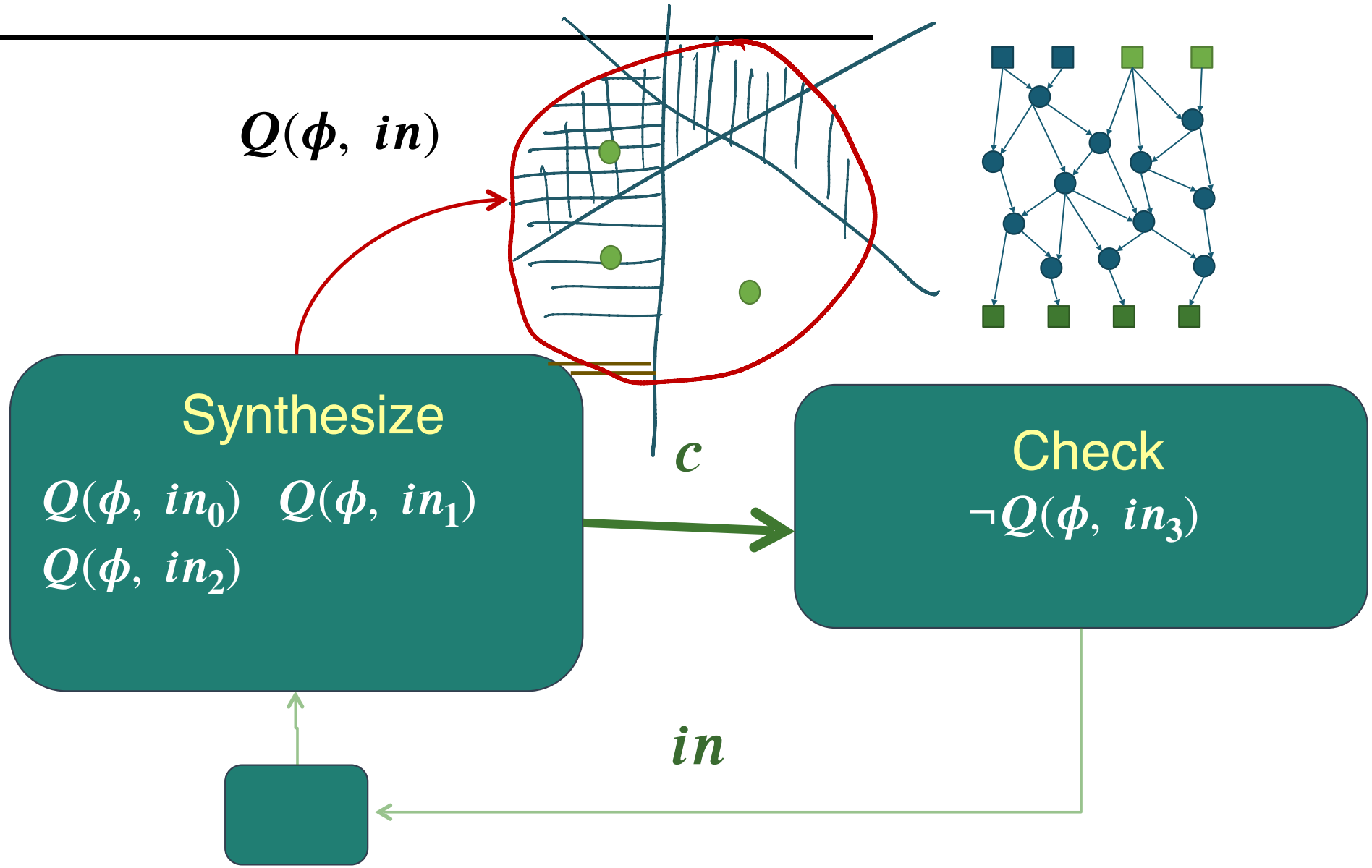
Constraint-based CEGIS



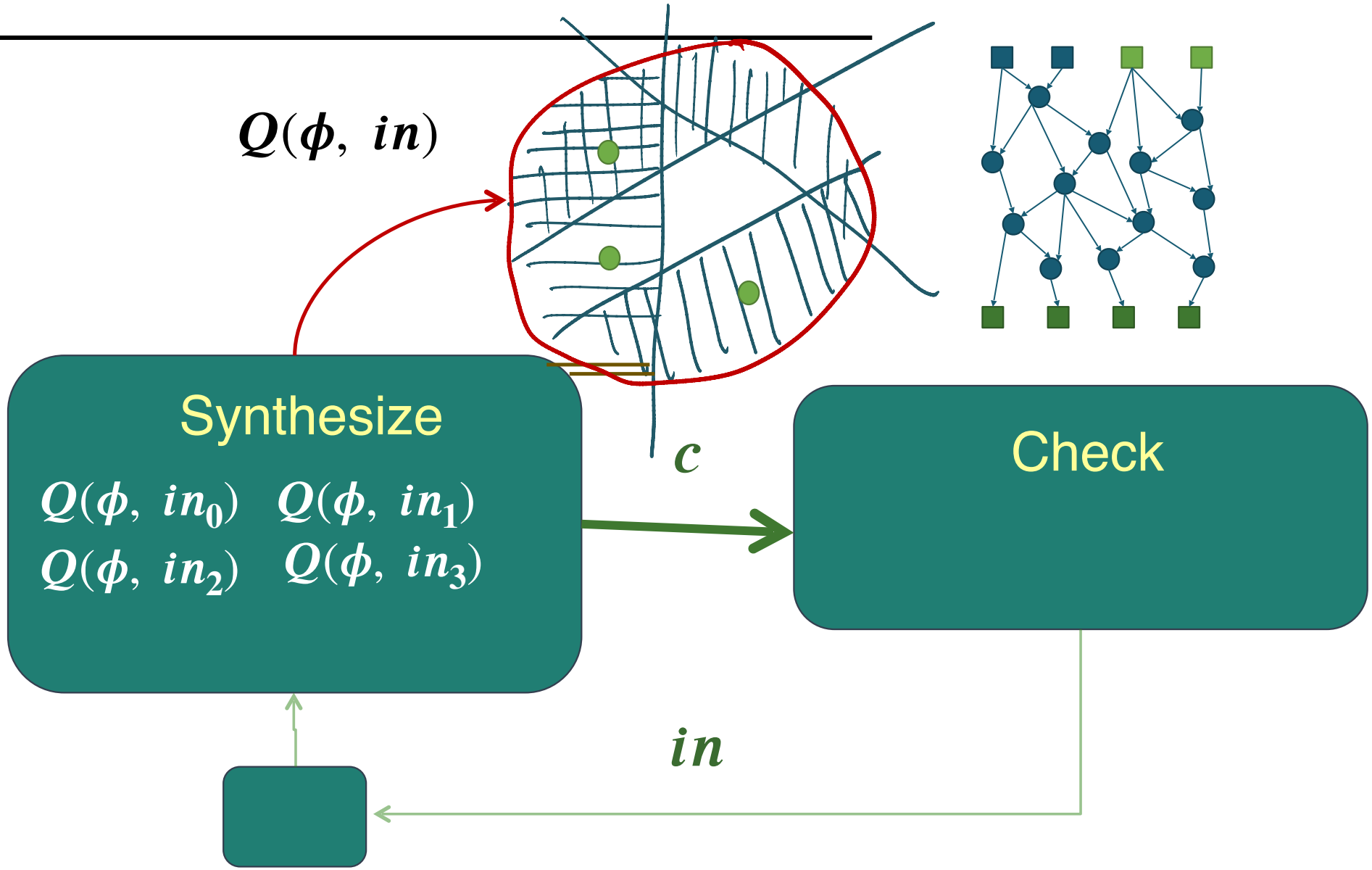
Constraint-based CEGIS



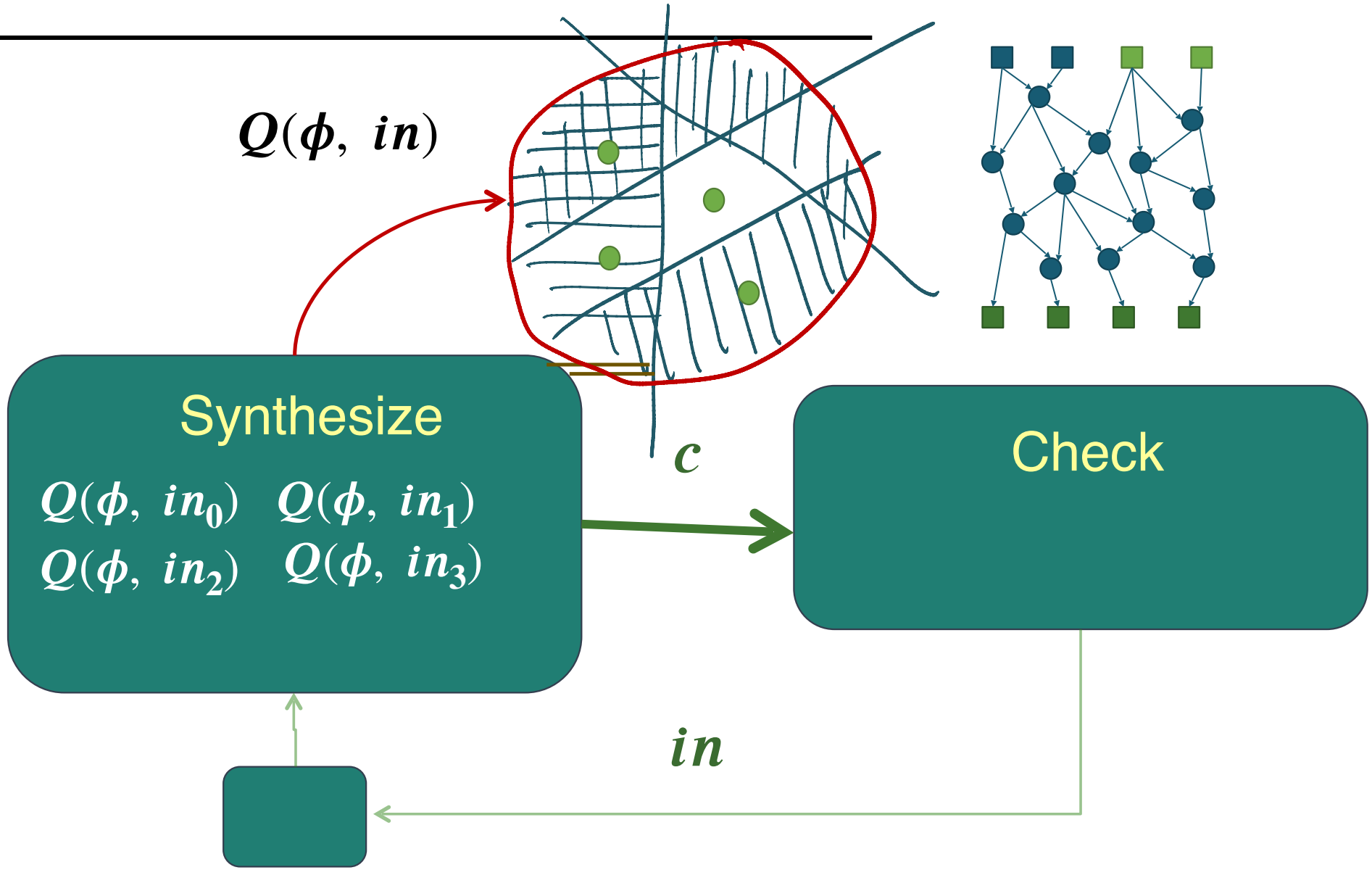
Constraint-based CEGIS



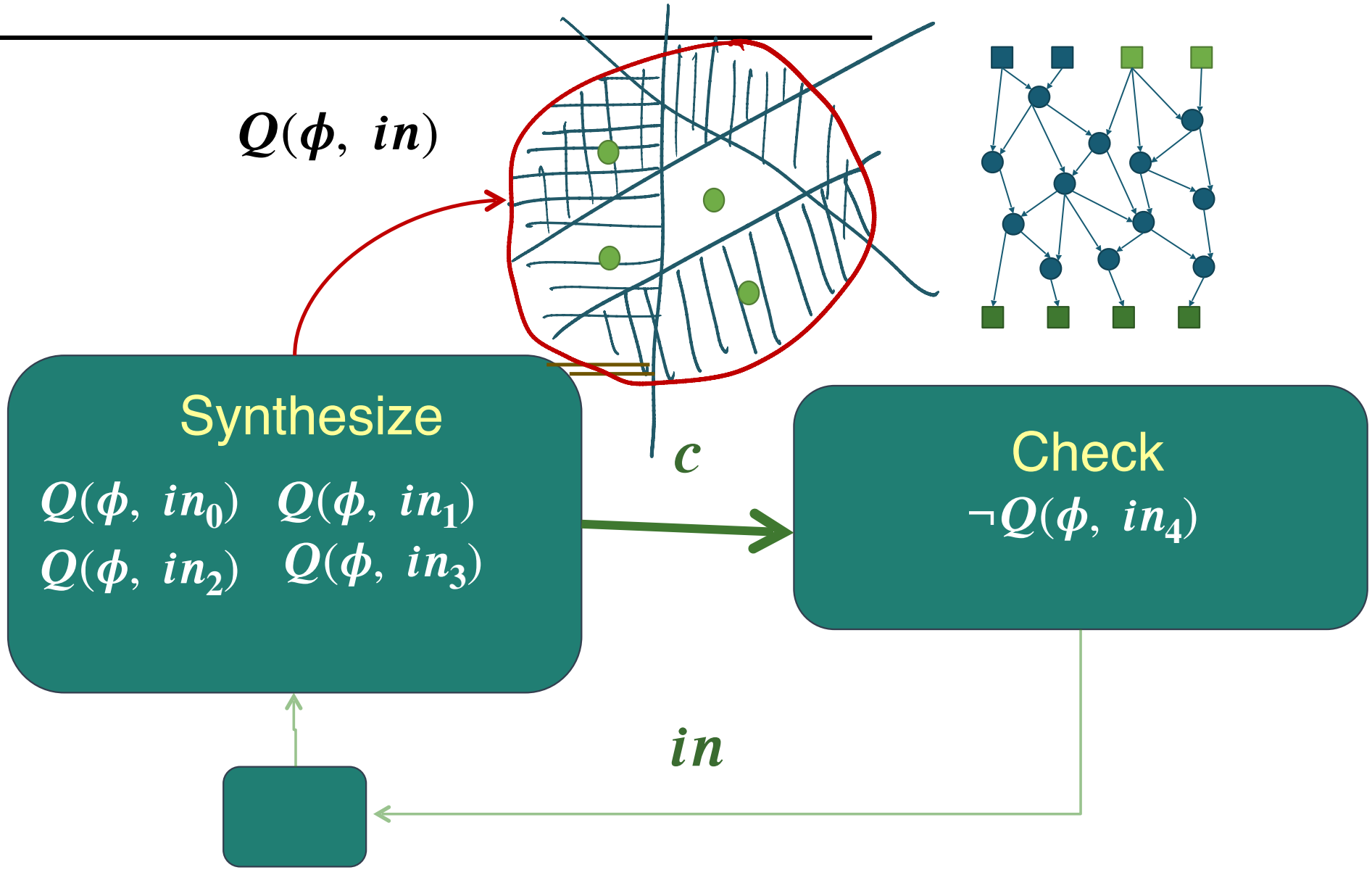
Constraint-based CEGIS



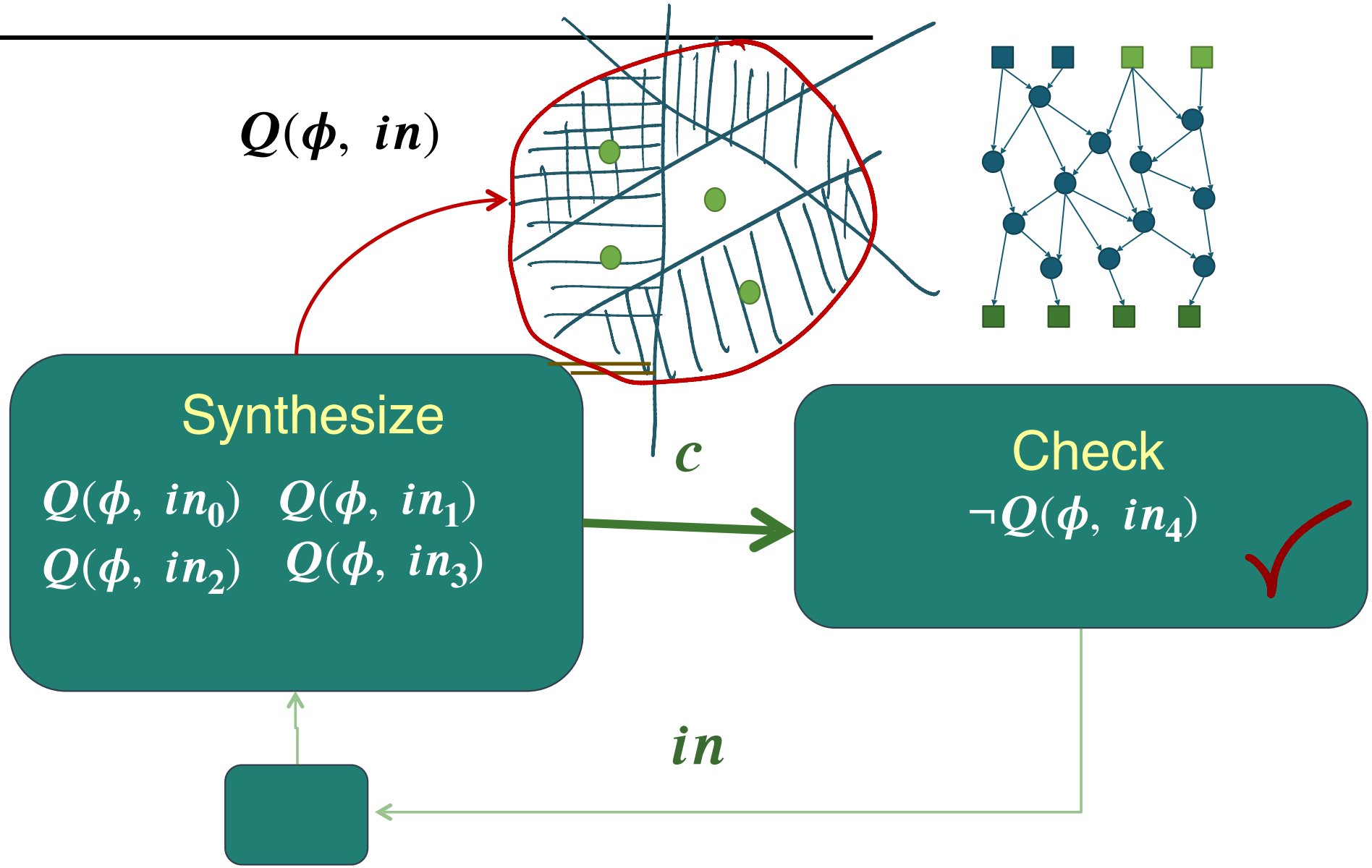
Constraint-based CEGIS



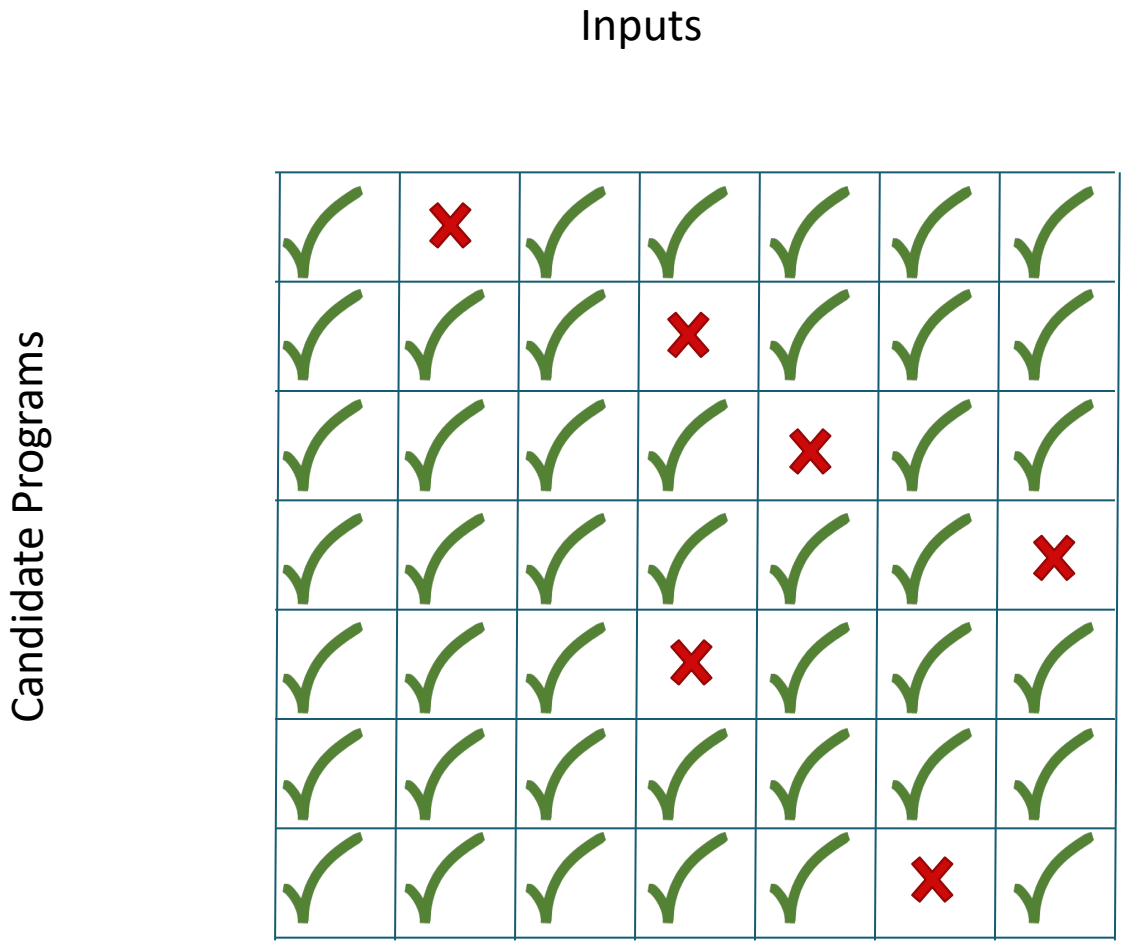
Constraint-based CEGIS



Constraint-based CEGIS



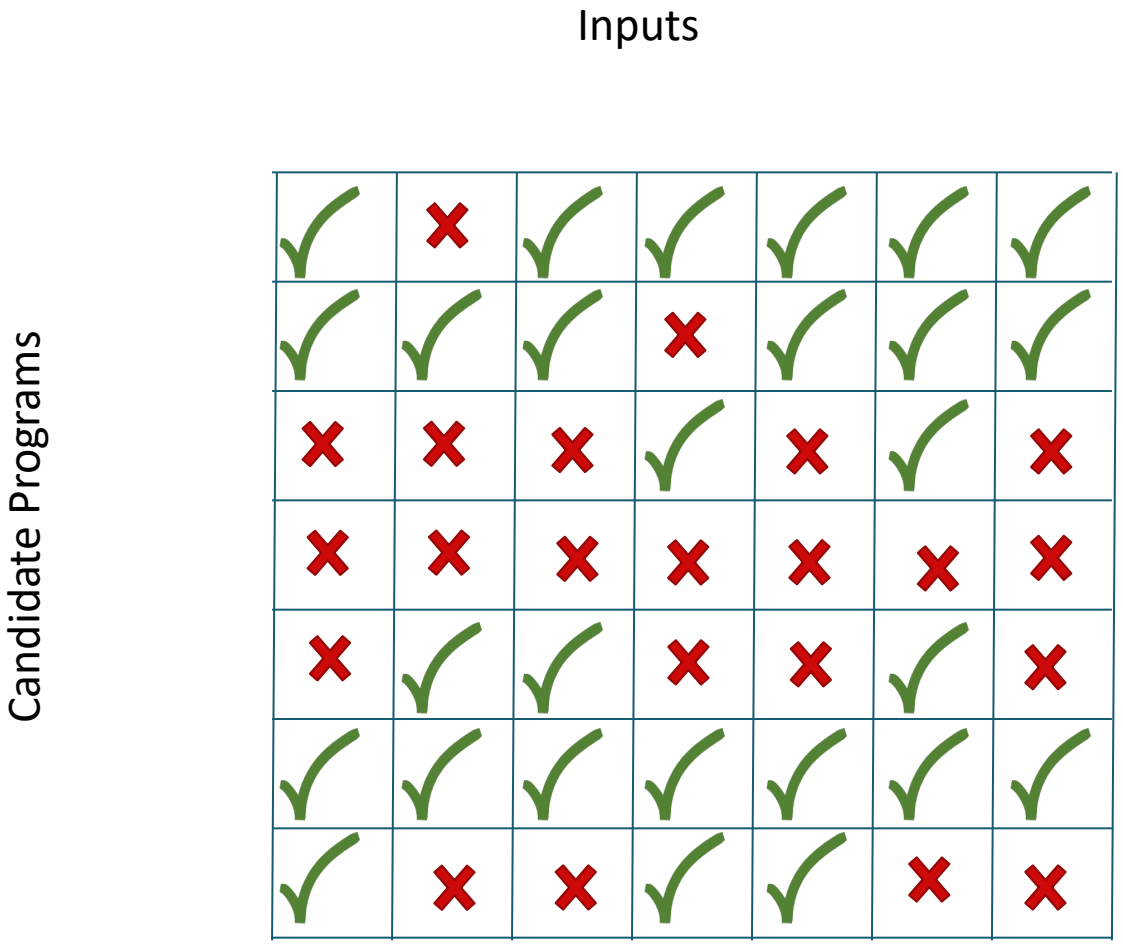
CEGIS



Most candidates work for most inputs.

This is a problem for CEGIS because each counterexample input eliminates very few candidates

CEGIS



CEGIS works well
in the case where aside from
a few almost-correct programs,
most buggy programs will fail
on large sets of inputs, so each
counterexample can eliminate
many programs.

Abstraction

Use abstract objects to represent sets of inputs

Propagate abstract objects through the program

Extract constraints on ϕ from the resulting abstract values

```
foo(x,y)
  t = y*φ(??1)
  v = x*φ(??2)
  assert t > v;
}
```

x = [1, 10]

y = [1, 10]

⇩

t = [ϕ_1, 10*ϕ_1]

v = [ϕ_2, 10*ϕ_2]

so $\phi_1 > 10\phi_2 \Rightarrow t > v$

Beyond Constraints

The constraint-based approach to verification works well but is limited by the capabilities of the verifier

Incorporation of deductive reasoning can help!

We will explore two additional directions

- Synthesis from Refinement Types
- Deductive Synthesis
- Hybrids

Synthesis from refinement types

Polikarpova, Kuraj, Solar-Lezama PLDI 2015

replicate :: n: Int → x: β → v: List β

replicate :: n: Nat → x: β → { v: List β | len v = n }

Example: insert

Goal type:

x: β → xs: IncList β → { v: IncList β | elems v = elems xs + [x] }

Components:

data IncList α **where**

Nil :: : IncList | }

Cons :: x: α → xs: IncList { v: α | v ≥ x }

→ : IncList [x]}

Insert solution

Goal type:

x: β → xs: IncList β → { v: IncList β | elems v = elems xs + [x] }

Solution:

```
insert x xs = match xs with
  Nil → Cons x Nil
  Cons y ys → if x ≤ y
    then Cons x (Cons y ys)
    else Cons y (insert x ys):: IncList { v: Int | v ≥ y }
```